

Widgets

Są to obiekty w Pythonie reagujące na zdarzenia i umożliwiające obsługę różnych popularnych kontroltek w przeglądarce. Wykorzystane zostaną do utworzenia prostego GUI do obsługi robota. Do ich obsługi konieczne jest zaimportowanie biblioteki ipywidgets.

```
In [1]: 1 import ipywidgets as widgets # zamiast korzystania z pełnej
        2 # nazwy biblioteki ipywidgets wykorzystany zostanie alias o
        3 # nazwie widgets.
        4
```

```
In [2]: 1 import rospy
        2 from geometry_msgs.msg import Twist
        3
```

```
In [3]: 1 rospy.init_node("ipywidgets_controller")
        2
```

Na zajęciach w których omawiany był Publisher wysyłanie prędkości sterujących odbywało się w następujący sposób:

```
In [4]: 1 # obiekt Publisher
        2 twist_publisher= rospy.Publisher("/turtle1/cmd_vel",Twist,
        3                               queue_size=10)
        4
```

```
In [5]: 1 # wysłanie wiadomości
        2 some_message=Twist()
        3 some_message.angular.z=1
        4 some_message.linear.x=1
        5
        6 twist_publisher.publish(some_message)
        7
```

```
In [6]: 1 # Można też utworzyć funkcję, która będzie przyjmowała argumenty
        2 # związane z prędkością postępową i obrotową robota i wysyłała do
        3 # ROS odpowiednią komendę na topic /turtle1/cmd_vel. Funkcja
        4 # wykorzystuje utworzony wcześniej obiekt Publisher o nazwie
        5 # twist_publisher.
        6 def move_robot(forward_vel=5,rotation_vel=5):
        7     '''A function to move turtle from turtlesim simulation
        8
        9     Args:
        10         forward_vel (float): Linear velocity
        11         rotation_vel (float): Angular velocity'''
        12     message=Twist()
        13     message.angular.z=rotation_vel
        14     message.linear.x=forward_vel
        15
        16     twist_publisher.publish(message)
        17
```

```
In [7]: 1 # wywołanie funkcji z przekazaniem argumentów
        2 move_robot(1,-1)
        3
```

Z punktu widzenia użytkownika konieczne jest dodanie kontroltek umożliwiającychysterowanie prędkości robota bez znajomości programowania.

Slider

Wykorzystamy funkcję **move_robot** do utworzenia domyślnego widgetu sterującego prędkościami robota. Każda zmiana wartości kontrolki powoduje wywołanie funkcji **move_robot** i wysłanie odczytanych prędkości z kontroltek. Argumenty funkcji dla, której tworzone będą kontrolki powinny mieć przypisane domyślne wartości. Dla tej funkcji zainicjalizowane domyślnie zostaną dwie kontrolki typu IntSlider. Zakres wartości również jest domyślny.

Do utworzenia kontroltek wykorzystywana jest funkcja **interact** z biblioteki ipywidgets (używamy aliasu widgets). Jako argument przekazywana jest funkcja **move_robot**

```
In [8]: 1 widgets.interact(move_robot)
        2
```

forward_vel  5
rotation_vel  5

```
Out[8]: <function __main__.move_robot(forward_vel=5, rotation_vel=5)>
```

Sterując prędkościami robota, które są w m/s skokowa zmiana o wartości typu Integer nie są pożądane. Przydałby się typ zmiennoprzecinkowy. W trakcie tworzenia widgetów można określić oddzielnie dla każdego argumentu funkcji typ kontrolki jaki ma obsługiwać. W tym przypadku zamienimy slidery typu FloatSlider, które umożliwiają ustawianie wartości zmiennoprzecinkowych.

Pierwszy argument przy wywołaniu funkcji widgets.interact pozostaje bez zmian, jeśli chodzi o nazwę funkcji. Kolejne argumenty jakie podajemy to nazwa argumentu z funkcji i przypisujemy do niej nową kontrolkę.

```
forward_vel=widgets.FloatSlider(min=-10,max=10,value=0)
```

Powyższy zapis oznacza, że dla argumentu **forward_vel** z funkcji **move_robot** utworzony zostanie **FloatSlider** z wartością minimalną -10 i maksymalną 10 oraz wartością początkową 0 pomimo tego, że w funkcji **move_robot** ta wartość wynosi 5.

```
In [9]: 1 widgets.interact(move_robot,
        2                 forward_vel=widgets.FloatSlider(min=-10,max=10,
        3                                                    value=0),
        4                 rotation_vel=widgets.FloatSlider(min=-3,max=3,
        5                                                    value=0))
        6
```

forward_vel  0.00
rotation_vel  0.00

```
Out[9]: <function __main__.move_robot(forward_vel=5, rotation_vel=5)>
```

```
In [10]: 1 # Dokumentację kontrolki można wyświetlić w jupyter notebook
        2 # po wpisaniu typu kontrolki do funkcji help
        3 help(widgets.FloatSlider)
        4
```

Help on class FloatSlider in module ipywidgets.widgets.widget_float:

```
class FloatSlider(_BoundedFloat)
|   Slider/trackbar of floating values with the specified range.
|
|   Parameters
|   -----
|   value : float
|       position of the slider
|   min : float
|       minimal position of the slider
|   max : float
|       maximal position of the slider
|   step : float
|       step of the trackbar
|   description : str
|       name of the slider
|   orientation : {'horizontal', 'vertical'}
|       default is 'horizontal', orientation of the slider
|   ...
```

Dla różnych argumentów kontrolki mogą być innego typu np. dla **rotation_vel** utworzona została kontrolka pozwalająca użytkownikowi na wprowadzenie wartości. Argument **step** dla kontrolki **FloatSlider** ustawia krok z jakim mają się zmieniać wartości w kontrolce.

```
In [11]: 1 widgets.interact(move_robot,
2                     forward_vel=widgets.FloatSlider(min=-10,max=10,
3                                                     step=2,value=0),
4                     rotation_vel=widgets.FloatText(value=0))
5
```

forward_vel  0.00
rotation_vel

```
Out[11]: <function __main__.move_robot(forward_vel=5, rotation_vel=5)>
```

Brak podania kontrolki dla argumentu powoduje utworzenie domyślnego slidera.

```
In [12]: 1 widgets.interact(move_robot,
2                     rotation_vel=widgets.FloatText(value=0))
3
```

forward_vel  5
rotation_vel

```
Out[12]: <function __main__.move_robot(forward_vel=5, rotation_vel=5)>
```

Textbox

Teraz czas na wysłanie dowolnej informacji na topic **/informacja**. Skorzystanie z kontrolki do wprowadzania tekstu spowoduje, że każde wprowadzenie nowego znaku wyśle wiadomość. W terminalu można podejrzeć informację na tym topicu.

rostopic echo /informacja

Oto przykład:

```
In [13]: 1 from std_msgs.msg import String
2 def send_msg(wiadomosc):
3     pub_info = pub_speed=rospy.Publisher("/informacja",String,
4                                         queue_size=10)
5     msg_info = String()
6     msg_info.data = wiadomosc
7     pub_info.publish(msg_info)
8
```

Tym razem przekazywana jest funkcja **send_msg** oraz uzupełniany jej argument **wiadomosc**.

```
In [14]: 1 widgets.interact(send_msg,
2                     wiadomosc=widgets.Text(value='Hello World!'))
3
```

wiadomosc

```
Out[14]: <function __main__.send_msg(wiadomosc)>
```

Utworzenie kontrolki bez wartości domyślnej dla tekstu. Wartość jest pustym tekstem.

```
In [15]: 1 widgets.interact(send_msg,
2                     wiadomosc=widgets.Text())
3
```

wiadomosc

```
Out[15]: <function __main__.send_msg(wiadomosc)>
```

Odczyt wartości z kontrolki typu **FloatText**. Utworzony został obiekt **liczba** przechowujący dane z kontrolki.

```
In [17]: 1 liczba = widgets.FloatText(
2     value=7.5,
3     description='Any:',
4     disabled=False
```

```

5 )
6 odczytana_liczba = liczba.value # odczyt wartości z kontrolki
7 # i zapis do zmiennej
8 print(odczytana_liczba) # wyświetlenie odczytanej wartości
9 print("typ wartości: ", type(odczytana_liczba)) # wyświetlenie
10 # typu odczytanej liczby
11
7.5
typ wartości: <class 'float'>

```

In [18]:

```

1 liczba
2

```

Any:

In [19]:

```

1 liczba.value
2

```

Out[19]: 7.5

Button

W poprzednim przykładzie każdorazowa zmiana tekstu powoduje jego wysłanie. Z punktu widzenia użytkownika konieczne jest wprowadzenie całego tekstu, a następnie wysłanie go do użytkownika.

In [20]:

```

1 widgets.ToggleButton(
2     value=False, # domyślna wartość dla przycisku
3     description='Przykładowy przycisk', # opis przycisku
4     disabled=False, # początkowy stan przycisku - wyłączony
5     button_style='', # 'success', 'info', 'warning', 'danger'
6     # or ''
7     tooltip='Description', # Pojawiający się szczegółowy opis
8     # po najechnaniu na przycisk
9     icon='check' # (FontAwesome names without the `fa-` prefix)
10 )
11

```

✓ Przykładowy prz...

Teraz czas na modyfikację kodu do wysyłania informacji po wciśnięciu przycisku wyslij. W tym celu tworzona jest zmienna `input_text`. Do wyświetlenia kontrolki służy funkcja `display()`, a jako argument przekazywana jest zmienna przechowująca kontrolkę, którą chcemy wyświetlić. W tym przypadku `input_text`.

Teraz kontrolka z tekstem nie jest w żaden sposób zależna z funkcją od wysyłania tekstu.

In [21]:

```

1 input_text = widgets.Text(value='Hello World!', disabled=False,
2                           description="informacja:")
3 display(input_text)
4

```

informacja:

Odczyt wartości z kontrolki ze zmiennej `input_text` i zapisanie go do zmiennej `odczytany_tekst`, a następnie wyświetlana jest wartość i typ przechowywanej wartości.

In [22]:

```

1 odczytany_tekst = input_text.value
2 print(odczytany_tekst)
3 print("typ wartości: ", type(odczytany_tekst))
4

```

Hello World!
typ wartości: <class 'str'>

Funkcja `send_msg` jako argument przyjmuje informację o stanie przycisku `wyslij` opisanego poniżej (patrz `description`, zmienna `przycisk_wyslij`), z którego pochodzi żądanie wywołania tej funkcji po każdorazowym kliknięciu przycisku.

Funkcja `send_msg` wykorzystuje wartość tekstu odczytaną ze zmiennej od kontrolki tekstowej `input_text`.

```
In [23]: 1 from std_msgs.msg import String
2 def send_msg(button_data):
3     pub_info = pub_speed=rospy.Publisher("/informacja",String,
4                                           queue_size=10)
5     msg_info = String()
6     msg_info.data = input_text.value
7     pub_info.publish(msg_info)
8
```

Utworzenie przycisku do wysyłania wiadomości.

```
In [24]: 1 przycisk_wyslij = widgets.Button(
2     value=False,
3     description='wyslij',
4     disabled=False,
5     button_style='', # 'success', 'info', 'warning', 'danger'
6     # or ''
7     tooltip='Description',
8     icon='check' # (FontAwesome names without the `fa-` prefix)
9 )
10 display(przycisk_wyslij)
11
```

✓ wyslij

W tej chwili po wykonaniu podłączenia na topic'u informacja można zobaczyć, że wiadomość nie jest publikowana.

Przycisk wyslij został utworzony, ale brakuje jeszcze obsługi zdarzenia na kliknięcie. Dla utworzonego obiektu przycisku **przycisk_wyslij** wykorzystywana jest metoda **on_click**, która reaguje na kliknięcie przycisku i powoduje wykonanie funkcji **send_msg** podanej w argumencie. Wywołana funkcja w argumencie posiada informację związane z przyciskiem od którego pochodzi zdarzenie.

```
In [25]: 1 przycisk_wyslij.on_click(send_msg)
2
```

Dopiero po obsłudze zdarzenia na kliknięcie możliwe jest wysyłanie tekstu.

Checkbox

Przycisk zawierający tylko dwa stany True lub False. Przykład użycia

```
In [26]: 1 checkbox_object = widgets.Checkbox(
2     value=False,
3     description='Check me',
4     disabled=False,
5     indent=False
6 )
7 display(checkbox_object)
8
```

☐ Check me

```
In [27]: 1 checkbox_object.value
2
```

Out[27]: False

Dropdown

Lista rozwijana z opcjami do wyboru

```
In [28]: 1 dropdown = widgets.Dropdown(
2     options=['1', '2', '3'],
3     value='2',
4     description='Number:',
5     disabled=False,
6 )
7 display(dropdown)
```

8

Number:

```
In [29]: 1 # Odczyt wartości
         2 dropdown.value
         3
```

Out[29]: '2'

```
In [30]: 1 # Typ wartości
         2 type(dropdown.value)
         3
```

Out[30]: str

RadioButtons

Lista przycisków z opcją do wyboru

```
In [31]: 1 radio_button = widgets.RadioButtons(
         2     options=['A', 'B', 'C'],
         3     value='B', # Domyślna wartość
         4     # layout={'width': 'max-content'}, # If the items' names
         5     # are long
         6     description='Wersja testu:',
         7     disabled=False
         8 )
         9 display(radio_button)
        10
```

Wersja testu: ☐ A
☒ B
☐ C

```
In [32]: 1 # odczyt wartości z kontroli
         2 radio_button.value
         3
```

Out[32]: 'B'

```
In [33]: 1 type(radio_button.value)
         2
```

Out[33]: str

Wybór wielu opcji

Lista wyboru wielu opcji

```
In [34]: 1 options = widgets.SelectMultiple(
         2     options=['skanowanie', 'jazda autonomiczna',
         3             'rozpoznawanie piłek tenisowych'],
         4     value=['skanowanie'],
         5     #rows=10,
         6     description='Funkcjonalności robota',
         7     disabled=False
         8 )
         9 display(options)
        10
```

Funkcjonal...

```
In [35]: 1 # odczyt wartości
         2 options.value
```

```
3
Out[35]: ('skanowanie',)
```

```
In [36]: 1 # Do pola z opisem opcji przypisanie wartości
2 options2 = widgets.SelectMultiple(
3     options=[('skanowanie',7), ('jazda autonomiczna', 2),
4             ('rozpoznawanie piłek tenisowych', 3)],
5     value=[2,3], # zamiast nazw pola podane sa wartości opcji,
6     # które zostały wybrane
7     #rows=10,
8     description="Funkkcjonalności robota",
9     disabled=False
10 )
11 display(options2)
12
```

Funkkcjona...

skanowanie

jazda autonomiczna

rozpoznawanie piłek tenisowych

```
In [37]: 1 options2.value
2
```

```
Out[37]: (2, 3)
```

Pobieranie koloru

Kontrolka do wyboru koloru

```
In [38]: 1 color_picker = widgets.ColorPicker(
2         concise=False,
3         description='Pick a color',
4         value='blue',
5         disabled=False
6     )
7 display(color_picker)
8
```

Pick a color

blue

```
In [39]: 1 # wartość koloru zapisana jako typ tekstowy str w wartości
2 # koloru podanego hexadecymalnie
3 color_picker.value
4
```

```
Out[39]: 'blue'
```

Tabs

Do grupowania zakładek i wyświetlania wszystkich w jednej karcie służy **VBox** z omawianej biblioteki. Przyjmuje listę kontroltek do wyświetlenia.

```
In [40]: 1 tab_contents = ['P0', 'P1', 'P2']
2 # Ustawienie kontroltek w kolejnych zakładkach. Dla zakładek
3 # tekstowych nadawana jest nazwa taka jak karty. Zmienna children
4 # zawiera listę z 3 obiektami VBox, w którym każdy z nich zawiera
5 # 3x IntSlider oraz Text
6 children = [widgets.VBox([widgets.Text(description=name),
7                           widgets.IntSlider(), widgets.IntSlider(),
8                           widgets.IntSlider()])
9             for name in tab_contents]
10 tab = widgets.Tab(children = children)
11
12 # Ustawienie nazw kolejnych zakładek
13 for i in range(len(tab_contents)):
14     tab.set_title(i, tab_contents[i])
```

```

15 # wyświetlenie zakładek
16 tab
17

```

P0	P1	P2
P0		
<input type="range"/>		0
<input type="range"/>		0
<input type="range"/>		0

In [41]:

```

1 # Każdą z kontrolkek można zdefiniować oddzielnie, a następnie
2 # można je dodać do odpowiednich zakładek
3 kontrolka1 = widgets.IntSlider(description="speed")
4 kontrolka1_2 = widgets.Checkbox(description="slider_source")
5 kontrolka2 = widgets.IntSlider(description="cos tam")
6 kontrolka3 = widgets.Text(description="info")
7 kontrolka3_2 = widgets.IntSlider(description="forward")
8 kontrolka3_3 = widgets.IntSlider(description="rotational")
9
10 # lista z zakładkami
11 children = [
12     widgets.VBox([kontrolka1, kontrolka1_2]), # pierwsza zakładka,
13     # zawiera kontrolka1 i kontrolka1_2
14     widgets.VBox([kontrolka2]), # druga zakładka zawiera tylko
15     # jedną kontrolkę kontrolka2
16     widgets.VBox([kontrolka3, kontrolka3_2, kontrolka3_3])
17 ]
18
19 tab = widgets.Tab(children = children)
20 # ustawianie nazwy zakładek; Kolejne argumenty: id zakładki,
21 # nazwa zakładki
22 tab.set_title(0, "P1")
23 tab.set_title(1, "P2")
24 tab.set_title(2, "P3")
25 tab
26

```

P1	P2	P3
speed <input type="range"/> 0 ☐ slider_source		

Dodatkowe widgety

<https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20List.html> (<https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20List.html>)

Zmiana stylu i układu kontrolkek

Do opisu wyglądu i stylu kontrolkek służy obiekt `ipywidgets.Layout`.

In [42]:

```

1 ## Sprawdzanie dostępnych stylów dla kontrolki
2 b1=widgets.Button(description='To jest przycisk',
3                    layout=widgets.Layout(width='50%',
4                                           height='60px'))
5 b1.style.keys
6

```

Out[42]:


```
['_model_module_version',  
 '_view_module',  
 '_view_count',
```

Definiowanie rozmiaru

Na etapie inicjalizacji kontrolki jako argument layout należy przekazać obiekt `ipywidgets.Layout` z podanym rozmiarem kontrolki. W tym przypadku jest to 50% szerokości oraz 60px wysokości. Można też przekazać argumenty dotyczące maksymalnych i minimalnych rozmiarów.

Dostępne parametry:

- `height`,
- `width`,
- `max_height`,
- `max_width`,
- `min_height`,
- `max_height`

```
In [43]: 1 przycisk = widgets.Button(description='To jest przycisk',  
2         layout=widgets.Layout(width='50%', height='60px'))  
3 display(przycisk)  
4 # przycisk zajmuje 50% szerokości obszaru  
5
```

To jest przycisk

Ustawienie siatki do pozycjonowania elementów

```
In [44]: 1 button_layout = widgets.Layout(width='auto', height='auto')  
2 button_style = widgets.ButtonStyle(button_color='darkseagreen')  
3 children = [widgets.Button(layout=button_layout,  
4         style=button_style) for i in range(20)]  
5 grid = widgets.GridBox(children=children, # ustawienie kontrolki  
6 #         które będą kolejno wpisane do siatki  
7         layout=widgets.Layout(  
8             width='50%',  
9             # ustawienie szerokości kolejnych kolumn  
10            grid_template_columns='100px 50px 100px 40px',  
11            # ustawienie szerokości kolejnych wierszy, rozmiar  
12            # dalszych wierszy jest automatyczny  
13            grid_template_rows='80px auto 80px 60px',  
14            # odstęp w pikselach pomiędzy wierszami,  
15            # a później kolumnami  
16            grid_gap='5px 10px')  
17        )  
18 display(grid)  
19
```

```
In [45]: 1 # wysyłanie różnych wiadomości tekstowych
2
3 from std_msgs.msg import String
4 def send_msg(wiadomosc):
5     pub_info = rospy.Publisher("/informacja",String,queue_size=10)
6     msg_info = String()
7     msg_info.data = wiadomosc
8     pub_info.publish(msg_info)
9
```

```
In [46]: 1 layout = widgets.Layout(width='auto', height='auto')
2 text1 = widgets.Text(value='Tekst 1', layout=layout)
3 text2 = widgets.Text(value='Tekst 2', layout=layout)
4 text3 = widgets.Text(value='Tekst 3', layout=layout)
5 text4 = widgets.Text(value='Tekst 4', layout=layout)
6 text5 = widgets.Text(value='Tekst 5', layout=layout)
7
8
9 widgets.interact(send_msg, wiadomosc=text1)
10 widgets.interact(send_msg, wiadomosc=text2)
11 widgets.interact(send_msg, wiadomosc=text3)
12 widgets.interact(send_msg, wiadomosc=text4)
13 widgets.interact(send_msg, wiadomosc=text5)
14
```

wiadomosc

wiadomosc

wiadomosc

wiadomosc

wiadomosc

Out[46]: <function __main__.send_msg(wiadomosc)>

```
In [48]: 1 children_data=[text1, text2, text3, text4, text5]
2 grid = widgets.GridBox(children=children_data,
3     layout=widgets.Layout(
4         width='50%',
5         grid_template_columns='150px 150px 150px',
6         grid_template_rows='80px auto 80px 60px',
7         grid_gap='5px 10px')
8     )
9 display(grid)
10
```

wiadomosc wiadomosc wiadomosc

wiadomosc wiadomosc

Zmiana kolorów przycisku

```
In [49]: 1 b1 = widgets.Button(description='Custom color')
2 b1.style.button_color = 'lightgreen'
3 display(b1)
4
```

Custom color

```
In [ ]: 1
```

Dodatkowe przykłady

<https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20Styling.html> (<https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20Styling.html>)

```
In [ ]: 1
```